SAVEETHA SCHOOL OF ENGINEERING

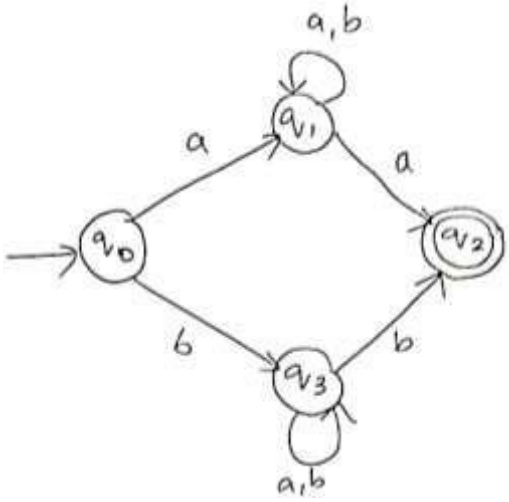
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

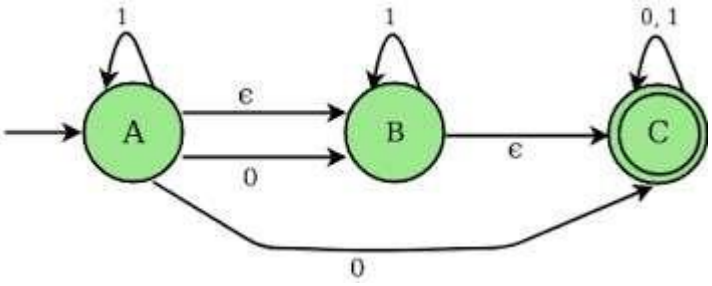
COMPUTER SCIENCE AND ENGINEERING

CSA1311 – THEORY OF COMPUTATION

ASSIGNMENT – 1

Name: DHIILIPAN M **REG NO:** 192411038

		CO	BL
1	Design a Deterministic Finite Automata (DFA) to accept strings that start and end with same symbols over **Σ**={0,1}. Write the formal definition of the DFA. Also show that the string **010100** is accepted by the DFA	1	3
2	Design NFA to accept string having 101 as substring	1	3
3	Apply the principle of mathematical induction and prove that for all positive integers n, the statement given below is true. $\sum_{i=1}^{n} i(i+1) = \frac{n(n+1)(n+2)}{3}$	1	1
4	Consider the NFA given below which accepts the language representing the set of all strings that start and end with the same symbol over the alphabet **Σ**={a,b}. Construct a DFA that represents the same language 	1	3
5	A NFA with ε-transitions is given below. Find ε-closure for each state and construct an equivalent NFA without ε-transitions that accepts the same language. Draw its transition table and mark the initial and final states	1	3


1
1
0, 1
A
B
C
ϵ
0
ϵ
0

SOLUTIONS

Answer 1: DFA for strings that start and end with the same symbol

Idea. The DFA has to remember two things while it reads the input: the *first* symbol of the string and the *last* symbol read so far. The first symbol never changes after it is read, so it decides which half of the machine we stay in. Inside each half only the latest symbol matters, and the string is accepted exactly when the latest symbol equals the first one. A string of length 1 such as 0 or 1 starts and ends with the same symbol, so it is accepted. The empty string has no first symbol, so it is rejected.

Meaning of each state

State	Meaning	Final?
q0	Start state, nothing read yet	No
q1	String began with 0 and the last symbol read is 0	Yes
q2	String began with 0 and the last symbol read is 1	No
q3	String began with 1 and the last symbol read is 1	Yes
q4	String began with 1 and the last symbol read is 0	No

Table 1: Role of each state of the DFA

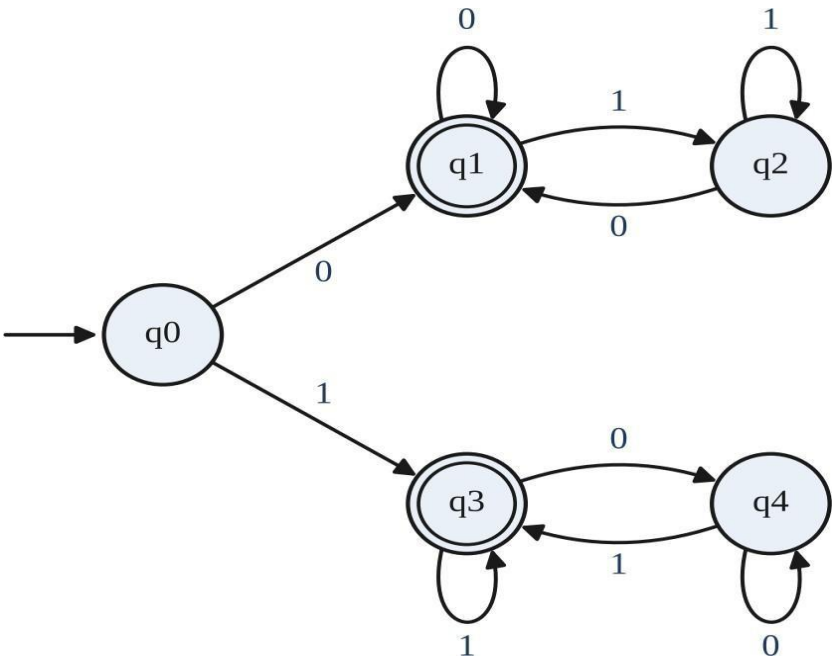


Figure 1: DFA accepting strings that start and end with the same symbol over {0, 1}

Formal definition

The DFA is the 5-tuple $M = (Q, \Sigma, \delta, q0, F)$ where:

- $Q = \{q0, q1, q2, q3, q4\}$
- $\Sigma = \{0, 1\}$
- Start state = q0
- $F = \{q1, q3\}$
- $\delta : Q \times \Sigma \to Q$ is given by the transition table below.

δ	0	1
→ q0	q1	q3
***** **q1**	q1	q2
q2	q1	q2
***** **q3**	q4	q3
q4	q4	q3

Table 2: Transition table (→ marks the start state, * marks a final state)

Acceptance of the string 010100

Starting in q0 and reading one symbol at a time, the extended transition function gives:

Step	Current state	Symbol read	Next state
1	q0	0	q1
2	q1	1	q2
3	q2	0	q1
4	q1	1	q2
5	q2	0	q1
6	q1	0	q1

Table 3: Trace of the DFA on 010100

As a path: q0 –0→ q1 –1→ q2 –0→ q1 –1→ q2 –0→ q1 –0→ q1. The DFA stops in q1, which belongs to F, so **010100 is accepted**. This agrees with the language, since the string starts with 0 and ends with 0.

Answer 2: NFA for strings containing 101 as a substring

Idea. Non-determinism lets the machine "guess" where the substring 101 starts. State q0 keeps reading any symbols while waiting. When a 1 appears, the NFA may *choose* to move to q1 (guessing that this 1 is the first symbol of 101). After that it must see 0 and then 1, and once it reaches q3 the substring has been found, so it stays there for the rest of the input. If the guess is wrong, that branch simply dies and the other branch (which stayed in q0) goes on.

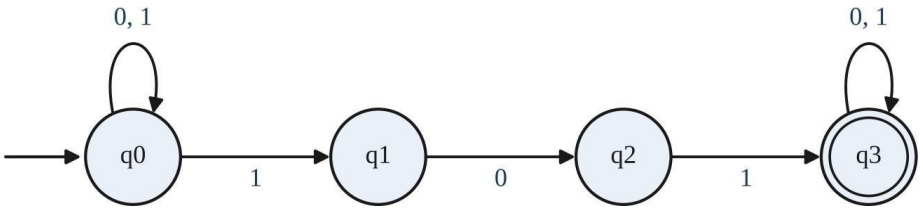


Figure 2: NFA accepting all strings over {0, 1} that contain 101 as a substring

Formal definition

N = (Q, Σ, δ, q0, F) with Q = {q0, q1, q2, q3}, Σ = {0, 1}, start state q0 and F = {q3}. The transition function is:

δ	0	1
→ q0	{q0}	{q0, q1}
q1	{q2}	{ }
q2	{ }	{q3}
*** q3**	{q3}	{q3}

Table 4: Transition table of the NFA

Example: the string 01101 is accepted

The string contains 101 at positions 3 to 5. The accepting branch of the computation is:

Symbol read	Branches alive after reading it
(start)	{q0}
0	{q0}
1	{q0, q1}
1	{q0, q1} (q1 on 1 dies; q0 gives q0 and q1 again)
0	{q0, q2}
1	{q0, q1, q3}

Table 5: Set of active states while reading 01101

The final set contains q3, which is a final state, so 01101 is accepted. In contrast, for 1100 the active sets are {q0}, {q0,q1}, {q0,q1}, {q0,q2}, {q0}; q3 is never reached, so 1100 is rejected, which is correct because 101 does not occur in it.

Answer 3: Proof by mathematical induction

Statement to prove. For every positive integer n,

$$P(n): 1\cdot2 + 2\cdot3 + 3\cdot4 + \ldots + n(n+1) = n(n+1)(n+2) / 3$$

The principle of mathematical induction says that if P(1) is true and P(k) implies P(k + 1) for every $k \geq 1$, then P(n) is true for all positive integers n.

Step 1: Basis step (n = 1)

LHS = 1(1 + 1) = 2.

RHS = $1\cdot(1 + 1)\cdot(1 + 2) / 3 = (1\cdot2\cdot3) / 3 = 2$.

LHS = RHS, so P(1) is true.

Step 2: Inductive hypothesis

Assume P(k) is true for some positive integer k, that is,

$$1\cdot2 + 2\cdot3 + \ldots + k(k + 1) = k(k + 1)(k + 2) / 3. \quad (1)$$

Step 3: Inductive step (to show P(k + 1))

We must show that

$$1\cdot2 + 2\cdot3 + \ldots + k(k + 1) + (k + 1)(k + 2) = (k + 1)(k + 2)(k + 3) / 3.$$

Start from the left-hand side and use (1) for the first k terms:

$$LHS = k(k + 1)(k + 2) / 3 + (k + 1)(k + 2)$$

Both terms have the common factor (k + 1)(k + 2), so take it out:

$$LHS = (k + 1)(k + 2) [k/3 + 1] = (k + 1)(k + 2) \cdot (k + 3) / 3$$

which is exactly the right-hand side of P(k + 1). Hence P(k) true gives P(k + 1) true.

Step 4: Conclusion

P(1) is true and P(k) → P(k + 1) for every $k \geq 1$. By the principle of mathematical induction, the formula holds for **all positive integers n**. Hence proved.

Numerical check

n	Sum of i(i+1) for i = 1 to n	n(n+1)(n+2)/3
1	2	2
2	2 + 6 = 8	$2\cdot3\cdot4/3 = 8$
3	8 + 12 = 20	$3\cdot4\cdot5/3 = 20$
4	20 + 20 = 40	$4\cdot5\cdot6/3 = 40$
5	40 + 30 = 70	$5\cdot6\cdot7/3 = 70$

Table 6: Verification for n = 1 to 5

Answer 4: Converting the given NFA to an equivalent DFA

Given NFA. From the start state q0, the symbol a leads to q1 and the symbol b leads to q3. State q1 loops on a, b and moves to the final state q2 on a. State q3 loops on a, b and moves to q2 on b. The first symbol is therefore remembered by the branch (q1 for a, q3 for b), and q2 is reached only when the same symbol appears again later.

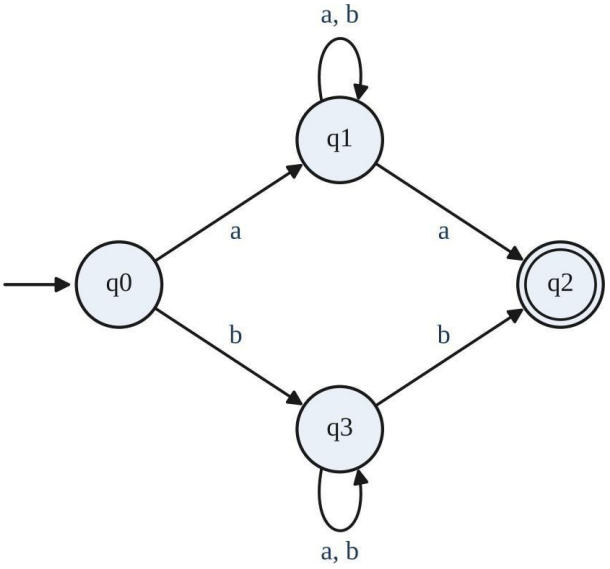


Figure 3: The given NFA (start q0, final q2)

δN	a	b
→ q0	{q1}	{q3}
q1	{q1, q2}	{q1}
q3	{q3}	{q2, q3}
*** q2**	{ }	{ }

Table 7: Transition table of the NFA

Subset construction

The DFA states are subsets of {q0, q1, q2, q3}. Starting from {q0}, only the subsets that can actually be reached are created. For a subset S, δD(S, x) is the union of δN(q, x) over all q in S.

DFA state	On a	On b
{q0}	δN(q0,a) = {q1}	δN(q0,b) = {q3}
{q1}	{q1, q2}	{q1}
{q3}	{q3}	{q2, q3}
{q1, q2}	δN(q1,a) union δN(q2,a) = {q1,q2}	δN(q1,b) union δN(q2,b) = {q1}
{q2, q3}	δN(q2,a) union δN(q3,a) = {q3}	δN(q2,b) union δN(q3,b) = {q2,q3}

Table 8: Subset construction (no new subsets appear after these five)

Renaming {q0} = A, {q1} = B, {q1,q2} = C, {q3} = D and {q2,q3} = E, the DFA is M = (Q, {a, b}, δ, A, F) with F = {C, E} (the subsets that contain the NFA final state q2). Figure 4 labels each state with its subset instead of the letter.

δD	a	b
→ **A**	B	D
B	C	B
* **C**	C	B
D	D	E
* **E**	D	E

Table 9: Transition table of the equivalent DFA

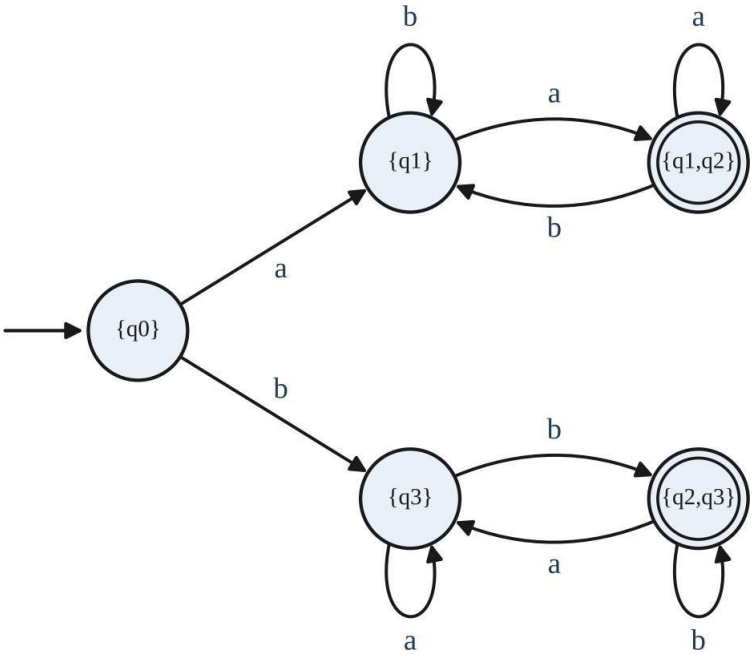


Figure 4: Equivalent DFA obtained by subset construction

Verification

For abba: A –a→ B –b→ B –b→ B –a→ C, and C is final, so it is accepted (starts and ends with a). For ab: A –a→ B –b→ B, which is not final, so it is rejected. For baab: A –b→ D –a→ D –a→ D –b→ E, which is final, so it is accepted. The DFA behaves exactly like the NFA on these strings.

Note. Every accepting path in the given NFA needs at least two symbols (q0 to q1 or q3, then at least one more symbol to reach q2). So the language of this NFA is the set of strings of length 2 or more that start and end with the same symbol, and the DFA above recognises exactly this language. A single-symbol string such as "a" is not accepted by the NFA as drawn, and therefore not by the DFA either.

Answer 5: Eliminating ε-transitions

Given ε-NFA. The start state is A and the final state is C. The transitions are: A on 1 stays in A, A on 0 goes to B and to C, and A goes to B on ε. State B loops on 1 and goes to C on ε. State C loops on both 0 and 1.

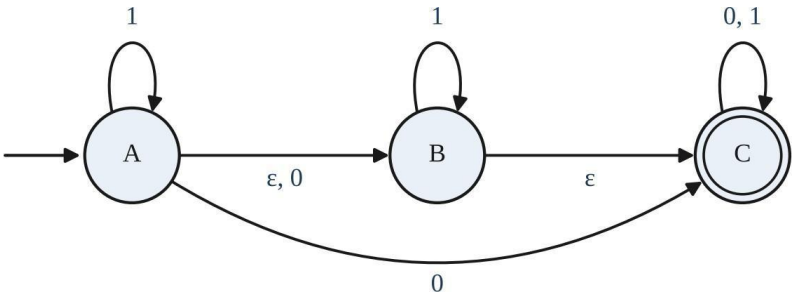


Figure 5: The given NFA with ε-transitions

δ	0	1	ε
→ **A**	{B, C}	{A}	{B}
B	{ }	{B}	{C}
***** **C**	{C}	{C}	{ }

Table 10: Transition table of the ε-NFA

Step 1: ε-closure of each state

The ε-closure of a state is the set of states reachable from it using only ε-transitions (including the state itself).

State	ε-closure	Reason
A	{A, B, C}	A –ε→ B and B –ε→ C
B	{B, C}	B –ε→ C
C	{C}	C has no ε-move

Table 11: ε-closures

Step 2: New transition function

The ε-free NFA uses δ'(q, x) = ε-closure(δ(ε-closure(q), x)). Computing each entry:

- **δ'(A, 0):** ε-closure(A) = {A, B, C}. δ on 0 gives A → {B, C}, B → { }, C → {C}, so the union is {B, C}. Its ε-closure is {B, C}.
- **δ'(A, 1):** δ on 1 gives A → {A}, B → {B}, C → {C}, so the union is {A, B, C}. Its ε-closure is {A, B, C}.
- **δ'(B, 0):** ε-closure(B) = {B, C}. δ on 0 gives B → { }, C → {C}, so the union is {C}. Its ε-closure is {C}.

- **δ'(B, 1):** δ on 1 gives B → {B}, C → {C}, so the union is {B, C}. Its ε-closure is {B, C}.
- **δ'(C, 0)** = {C} and **δ'(C, 1)** = {C}, since the closure of C is just {C}.

Step 3: Final states

A state becomes final in the new NFA if its ε-closure contains an original final state. The closures of A, B and C all contain C, so **A, B and C are all final**. In particular the empty string is accepted, because A can reach C by ε-moves alone. The start state stays A.

δ'	0	1
→ * **A**	{B, C}	{A, B, C}
* **B**	{C}	{B, C}
* **C**	{C}	{C}

Table 12: Transition table of the NFA without ε-transitions (→ initial, * final)

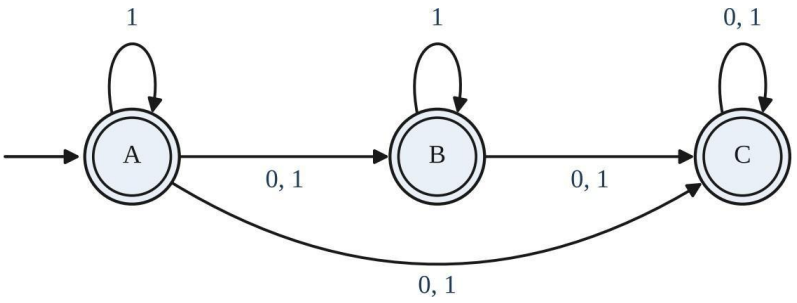


Figure 6: Equivalent NFA without ε-transitions (A is the initial state; A, B and C are final)

In Figure 6, the label "0, 1" on A to B means A goes to B on 0 (from δ'(A,0)) and on 1 (from δ'(A,1)); the same holds for A to C. The loop on A is on 1 only, because A is not in δ'(A, 0). State B goes to C on both symbols and loops on 1, and C loops on both symbols.

Check. The string 0 reaches C in the ε-NFA (A –0→ C) and in the new NFA (A –0→ C). The string 10 gives A –1→ A –0→ B or C in the new NFA, which is final, and in the original, A –1→ A –0→ C. Both machines accept it, so they recognise the same language.